COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	10/6/26	**DATE SUBMITTED:**	10/8/26
NAME:	Mike Justin Cabangon Denzen Igloso	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 11.1 **BASIC ALGORITHM ANALYSIS**

1. PROCEDURE OUTPUT

ILO A

Assess if two arrays have any common values.

```
bool diff(int *x, int *y){
    for(int i = 0; i < y.length; i++){
        if(search(x, y[i])!= - 1){
            return false;
        }
    }
    return true;
}
```

Performing worst case analysis:

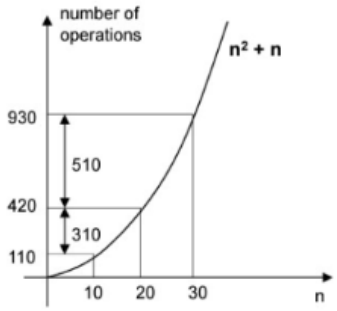
- Let m = the length of array x.
- Let n = the length of array y.
- The loop in diff repeats n times.
- Each call to search requires m comparisons.

wa

Given the above information, the total number of comparisons in the worst case is: **m * n**

- Assume that m = n. (The arrays are the same size)

Provide an analysis of the graph for the worse case of the algorithm.



n (size of array)	number of operations
10	110
20	420
30	930

Analysis:
From the graph, the number of operations increases as the array size n increases. For n = 10, there are 110 operations; for n = 20, there are 420 operations, and for n = 30, there are 930 operations. The graph forms a quadratic curve, following $n^2 + n$. Therefore, the worst-case time complexity of the algorithm is $O(n^2)$.

ILO B

Input Size (N)	Execution Speed	Screenshot	Observations
1,000	1 microseconds	`Time taken to search = 1 microseconds` `Student (417 716) needs vaccination.`	The algorithm completed the search at 1 microsecond.
10,000	0 microsecond	`Time taken to search = 0 microseconds` `Student (417 716) needs vaccination.`	Though it has a bigger input size it still executed quicker than the 1,000 input size
100,000	1 microseconds	`Time taken to search = 1 microseconds` `Student (417 716) needs vaccination.`	The input size is multiplied by 10 but executed at 1 microseconds

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1

Problem 1: Consider a program that returns true if the elements in an array are unique.

```
Algorithm Unique(A)
  for i = 0 to n-1 do
    for j = i+1 to n-1 do
      if A[i] equals to A[j] then
        return false
  return true
```

THEORETICAL ANALYSIS

The basic operation is the comparison A[i] == A[j]. In the worst case, no duplicate stops the loops early, so every pair is compared: (n−1) + (n−2) + … + 1 = n(n−1)/2, which is O(n²). In the best case, the first two elements are equal, so it returns false after 1 comparison, which is O(1). In the average case, about half of the pairs are checked before a duplicate is found, which is still O(n²). The space complexity is O(1).

EXPERIMENTAL ANALYSIS

CODE

```cpp
#include <iostream>
#include <iomanip>
#include <vector>
#include <chrono>

using namespace std;
using Clock = chrono::high_resolution_clock;

long long cmpCount = 0;

bool unique_(const vector<int>& A) {
    int n = A.size();
    for (int i = 0; i <= n - 2; i++)
        for (int j = i + 1; j <= n - 1; j++) {
            cmpCount++;
            if (A[i] == A[j]) return false;
        }
    return true;
}

int main() {
    cout << "Problem 1: Unique(A) - worst case (all elements unique)\n";
    cout << left << setw(10) << "n" << setw(18) << "comparisons"
         << setw(18) << "n(n-1)/2" << "time (ms)\n";
```

```cpp

    for (int n : {1000, 2000, 4000, 8000, 16000}) {
        vector<int> A(n);
        for (int i = 0; i < n; i++) A[i] = i;

        cmpCount = 0;
        auto t0 = Clock::now();
        unique_(A);
        auto t1 = Clock::now();
        double ms = chrono::duration<double, milli>(t1 - t0).count();

        cout << left << setw(10) << n << setw(18) << cmpCount
             << setw(18) << (long long)n * (n - 1) / 2 << ms << "\n";
    }

    vector<int> B(16000, 0);
    cmpCount = 0;
    unique_(B);
    cout << "\nBest case (A[0] == A[1]), n = 16000: " << cmpCount << " comparison\n";
    return 0;
}
```

OUTPUT

n	comparisons	n(n-1)/2	time (ms)
1000	499500	499500	11.4236
2000	1999000	1999000	34.0399
4000	7998000	7998000	69.8066
8000	31996000	31996000	272.299
16000	127992000	127992000	1144.48

ANALYSIS AND COMPARISON

The experimental results matched the theoretical analysis. At n = 16,000, the program made exactly n(n−1)/2 = 127,992,000 comparisons, and each time n doubled, the runtime grew about 4×, from 143 ms at n = 8,000 to 571 ms at n = 16,000. This confirms that Unique(A) runs in $O(n^2)$ time in the worst case.

Problem 2

```
Algorithm rpower(int x, int n):
1 if n == 0 return 1
2 else return x*rpower(x, n-1)

Algorithm brpower(int x, int n):
1 if n == 0 return 1
2 if n is odd then
3   y = brpower(x, (n-1)/2)
4   return x*y*y
5 if n is even then
6   y = brpower(x, n/2)
7   return y*y
```

THEORETICAL ANALYSIS

The basic operation is multiplication, and the count depends only on n, so there are no separate best, worst, or average cases.

rpower makes one multiplication per call and recurses on n−1, so T(n) = T(n−1) + 1, which gives T(n) = n. The time is O(n) and the space is O(n) because of the recursion depth.

brpower halves the exponent at each call, using 1 or 2 multiplications per level, so T(n) = T(⌊n/2⌋) + c. This gives about $\log_2 n$ levels, so the time is O(log n) and the space is O(log n).

brpower is much more efficient. At n = 1,000, rpower needs 1,000 multiplications while brpower needs only 16.

EXPERIMENTAL ANALYSIS

CODE

```cpp
#include <iostream>
#include <iomanip>
#include <chrono>

using namespace std;
using Clock = chrono::high_resolution_clock;

long long mulCount = 0;

double rpower(double x, int n) {
    if (n == 0) return 1;
    mulCount++;
    return x * rpower(x, n - 1);
}

double brpower(double x, int n) {
    if (n == 0) return 1;
    if (n % 2 == 1) {
        double y = brpower(x, (n - 1) / 2);
        mulCount += 2;
        return x * y * y;
    }
    double y = brpower(x, n / 2);
    mulCount += 1;
    return y * y;
}

```

```cpp
int main() {
    cout << "Problem 2: rpower vs brpower\n";
    cout << left << setw(10) << "n" << setw(14) << "rpower mults"
         << setw(16) << "brpower mults" << setw(16) << "rpower (us)"
         << "brpower (us)\n";

    const int REPS = 1000;
    volatile double sink;

    for (int n : {10, 100, 1000, 5000, 10000}) {
        mulCount = 0; rpower(1.0001, n);  long long r = mulCount;
        mulCount = 0; brpower(1.0001, n); long long b = mulCount;

        auto t0 = Clock::now();
        for (int k = 0; k < REPS; k++) sink = rpower(1.0001, n);
        auto t1 = Clock::now();
        for (int k = 0; k < REPS; k++) sink = brpower(1.0001, n);
        auto t2 = Clock::now();

        double tr = chrono::duration<double, micro>(t1 - t0).count() / REPS;
        double tb = chrono::duration<double, micro>(t2 - t1).count() / REPS;

        cout << left << setw(10) << n << setw(14) << r << setw(16) << b
             << setw(16) << tr << tb << "\n";
    }
    return 0;
}
```

OUTPUT

```
n         rpower mults   brpower mults   rpower (us)     brpower (us)
10        10             6               0.0235          0.0138
100       100            10              0.9787          0.1616
1000      1000           16              13.8867         0.0333
5000      5000           18              38.5225         0.046
10000     10000          19              77.106          0.0899
```

ANALYSIS AND COMPARISON

the experiment also agreed with the theory. rpower used exactly n multiplications and its runtime grew linearly with n, while brpower used only 19 multiplications at n = 10,000 and its runtime stayed almost flat. This confirms O(n) for rpower and O(log n) for brpower.

CONCLUSION

In this activity, We learned that an algorithm's efficiency depends on how its operation count grows with the input size n, and that theoretical and experimental analysis support each other. We derived the running times using summations and recurrences, then implemented the algorithms in C++ and compared the operation counts and

measured times against the theory. The results matched: Unique(A) grew quadratically, O(n²), rpower grew linearly, O(n), and brpower grew logarithmically, O(log n), which showed how much algorithm design matters.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Claude**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance, explanation of concepts.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.